Problem Statement:

Create a Recommender System to show personalized movie recommendations based on ratings given by a user and other users similar to them in order to improve user experience.

Data Dictionary:

RATINGS FILE DESCRIPTION

==========

All ratings are contained in the file "ratings.dat" and are in the following format:

UserID::MovieID::Rating::Timestamp

UserIDs range between 1 and 6040

MovieIDs range between 1 and 3952

Ratings are made on a 5-star scale (whole-star ratings only)

Timestamp is represented in seconds

Each user has at least 20 ratings

USERS FILE DESCRIPTION

==========

User information is in the file "users.dat" and is in the following format:

UserID::Gender::Age::Occupation::Zip-code

All demographic information is provided voluntarily by the users and is not checked for accuracy. Only users who have provided some demographic information are included in this data set.

Gender is denoted by a "M" for male and "F" for female

Age is chosen from the following ranges:

1: "Under 18"

18: "18-24"

25: "25-34"

35: "35-44"

45: "45-49"

50: "50-55"

56: "56+"

Occupation is chosen from the following choices:

0: "other" or not specified

1: "academic/educator"

2: "artist"

3: "clerical/admin"

4: "college/grad student"

5: "customer service"

6: "doctor/health care"

7: "executive/managerial"

8: "farmer"

9: "homemaker"

10: "K-12 student"

11: "lawyer"

12: "programmer"

13: "retired"

14: "sales/marketing"

15: "scientist"

16: "self-employed"

17: "technician/engineer"

18: "tradesman/craftsman"

19: "unemployed"

20: "writer"

MOVIES FILE DESCRIPTION

==========

Movie information is in the file "movies.dat" and is in the following format:

MovieID::Title::Genres

Titles are identical to titles provided by the IMDB (including year of release)

Genres are pipe-separated and are selected from the following genres:

Action

Adventure

Animation

Children's

Comedy

Crime

Documentary

Drama

Fantasy

Film-Noir

Horror

Musical

Mystery

Romance

Sci-Fi

Thriller

War

Western

Import Libraries:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
```

Set Options:

```
warnings.simplefilter('ignore')
pd.set_option("display.max_columns", None)
pd.options.display.float_format = '{:.2f}'.format
sns.set_style('white')
```

Download datasets:

```
users=pd.read_csv("zee-users.dat", encoding='ISO-8859-1')
users.head()
```

	UserID::Gender::Age::Occupation::Zip-code
0	1::F::1::10::48067
1	2::M::56::16::70072
2	3::M::25::15::55117
3	4::M::45::7::02460
4	5::M::25::20::55455

Next steps: Generate code with users | View recommended plots | New interactive sheet

```
users= users.rename(columns={'::1':'Gender','::2':'Age','::3':'Occupation','::4':'Zip-code'})
users.head()
```

	UserID::Gender::Age::Occupation::Zip-code
0	1::F::1::10::48067
1	2::M::56::16::70072
2	3::M::25::15::55117
3	4::M::45::7::02460
4	5::M::25::20::55455

Next steps: Generate code with users | View recommended plots | New interactive sheet

```
users = users['UserID::Gender::Age::Occupation::Zip-code'].str.split('::', expand=True)
users.columns = ['UserID', 'Gender', 'Age', 'Occupation', 'Zip-code']
users.head()
```

	UserID	Gender	Age	Occupation	Zip-code
0	1	F	1	10	48067
1	2	M	56	16	70072
2	3	M	25	15	55117
3	4	M	45	7	02460
4	5	M	25	20	55455

Next steps: Generate code with users | View recommended plots | New interactive sheet

```
users['Age'].unique()
```

```
array(['1', '56', '25', '45', '50', '35', '18'], dtype=object)
```

```
users.replace({'Age':{'1':  "Under 18",
                      '18':  "18-24",
                      '25':  "25-34",
                      '35':  "35-44",
                      '45':  "45-49",
                      '50':  "50-55",
                      '56':  "56 Above"}}, inplace=True)
```

```
users.replace({'Occupation':{'0': "other",
                             '1': "academic/educator",
                             '2': "artist",
                             '3': "clerical/admin",
                             '4': "college/grad student",
                             '5': "customer service",
                             '6': "doctor/health care",
                             '7': "executive/managerial",
                             '8': "farmer",
                             '9': "homemaker",
                             '10': "k-12 student",
                             '11': "lawyer",
                             '12': "programmer",
                             '13': "retired",
                             '14': "sales/marketing",
                             '15': "scientist",
                             '16': "self-employed",
                             '17': "technician/engineer",
                             '18': "tradesman/craftsman",
```

```
                        '19': "unemployed",
                        '20': "writer"}}, inplace=True)
```

```
users.head()
```

	UserID	Gender	Age	Occupation	Zip-code
0	1	F	Under 18	k-12 student	48067
1	2	M	56 Above	self-employed	70072
2	3	M	25-34	scientist	55117
3	4	M	45-49	executive/managerial	02460
4	5	M	25-34	writer	55455

Next steps: Generate code with users | View recommended plots | New interactive sheet

```
# Assuming 'ratings' DataFrame was created from "zee-ratings.dat" file
# and the columns are separated by '::'
ratings = pd.read_csv("zee-ratings.dat", delimiter='::', encoding='ISO-8859-1', header=None)
ratings.columns = ['UserID', 'MovieID', 'Rating', 'Timestamp']  # Assign column names
```

```
ratings.head()
```

	UserID	MovieID	Rating	Timestamp
0	UserID	MovieID	Rating	Timestamp
1	1	1193	5	978300760
2	1	661	3	978302109
3	1	914	3	978301968
4	1	3408	4	978300275

```
movies=pd.read_csv("zee-movies.dat",delimiter='::', encoding='ISO-8859-1', header=None)
movies.columns = ['MovieID', 'Title', 'Genres']
movies.head()
```

	MovieID	Title	Genres
0	Movie ID	Title	Genres
1	1	Toy Story (1995)	AnimationIChildren'slComedy
2	2	Jumanji (1995)	AdventureIChildren'slFantasy
3	3	Grumpier Old Men (1995)	ComedyIRomance
4	4	Waiting to Exhale (1995)	ComedyIDrama

Next steps: Generate code with movies | View recommended plots | New interactive sheet

```
df_1=pd.merge(movies, ratings, on='MovieID',how='inner')
df_1.head()
```

	MovieID	Title	Genres	UserID	Rating	Timestamp
0	1	Toy Story (1995)	AnimationIChildren'slComedy	1	5	978824268
1	1	Toy Story (1995)	AnimationIChildren'slComedy	6	4	978237008
2	1	Toy Story (1995)	AnimationIChildren'slComedy	8	4	978233496
3	1	Toy Story (1995)	AnimationIChildren'slComedy	9	5	978225952
4	1	Toy Story (1995)	AnimationIChildren'slComedy	10	5	978226474

```
data= pd.merge(df_1, users, on='UserID',how='inner')
data.head()
```

	MovieID	Title	Genres	UserID	Rating	Timestamp	Gender	Age	Occupation	Zip-code
0	1	Toy Story (1995)	AnimationIChildren'slComedy	1	5	978824268	F	Under 18	k-12 student	48067
1	1	Toy Story (1995)	AnimationIChildren'slComedy	6	4	978237008	F	50-55	homemaker	55117
2	1	Toy Story (1995)	AnimationIChildren'slComedy	8	4	978233496	M	25-34	programmer	11413

Performing EDA

```
data.shape
```

```
(1000209, 10)
```

No. of rows: 1000209

No. of columns: 10

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000209 entries, 0 to 1000208
Data columns (total 10 columns):
 #   Column      Non-Null Count    Dtype 
---  ------      --------------    ----- 
 0   MovieID     1000209 non-null  object
 1   Title       1000209 non-null  object
 2   Genres      1000209 non-null  object
 3   UserID      1000209 non-null  object
 4   Rating      1000209 non-null  object
 5   Timestamp   1000209 non-null  object
 6   Gender      1000209 non-null  object
 7   Age         1000209 non-null  object
 8   Occupation  1000209 non-null  object
 9   Zip-code    1000209 non-null  object
dtypes: object(10)
memory usage: 76.3+ MB
```

```
data.isna().sum()
```

	0
MovieID	0
Title	0
Genres	0
UserID	0
Rating	0
Timestamp	0
Gender	0
Age	0
Occupation	0
Zip-code	0

dtype: int64

There are no missing values

Feature Engineering:

```
data['Datetime'] = pd.to_datetime(data['Timestamp'],
                                  unit='s')
data
```

	MovieID	Title	Genres	UserID	Rating	Timestamp	Gender	Age	Occupation	Zip-code	Datet:
0	1	Toy Story (1995)	Animation|Children's|Comedy	1	5	978824268	F	Under 18	k-12 student	48067	2001- 23:37
1	1	Toy Story (1995)	Animation|Children's|Comedy	6	4	978237008	F	50-55	homemaker	55117	2000- 04:30
2	1	Toy Story (1995)	Animation|Children's|Comedy	8	4	978233496	M	25-34	programmer	11413	2000- 03:31
3	1	Toy Story (1995)	Animation|Children's|Comedy	9	5	978225952	M	25-34	technician/engineer	61614	2000- 01:25
4	1	Toy Story (1995)	Animation|Children's|Comedy	10	5	978226474	F	35-44	academic/educator	95370	2000- 01:34
...	...	...	...	...	...	...	...	...	...	...	

```
data['ReleaseYear'] = data['Title'].str.split().str[-1]
data.head()
```

	MovieID	Title	Genres	UserID	Rating	Timestamp	Gender	Age	Occupation	Zip-code	Datetime	Releas
0	1	Toy Story (1995)	Animation|Children's|Comedy	1	5	978824268	F	Under 18	k-12 student	48067	2001-01-06 23:37:48	
1	1	Toy Story (1995)	Animation|Children's|Comedy	6	4	978237008	F	50-55	homemaker	55117	2000-12-31 04:30:08	
		Toy									2000-12-	

```
data['ReleaseYear']= data['ReleaseYear'].str.lstrip('(').str.rstrip(")")
data.head()
```

	MovieID	Title	Genres	UserID	Rating	Timestamp	Gender	Age	Occupation	Zip-code	Datetime	Releas
0	1	Toy Story (1995)	Animation|Children's|Comedy	1	5	978824268	F	Under 18	k-12 student	48067	2001-01-06 23:37:48	
1	1	Toy Story (1995)	Animation|Children's|Comedy	6	4	978237008	F	50-55	homemaker	55117	2000-12-31 04:30:08	
		Toy									2000-12-	

```
data['Title'] = data['Title'].str.replace(r' \(\d{4}\)', '', regex=True)
data.head()
```

	MovieID	Title	Genres	UserID	Rating	Timestamp	Gender	Age	Occupation	Zip-code	Datetime	Releas
0	1	Toy Story	Animation|Children's|Comedy	1	5	978824268	F	Under 18	k-12 student	48067	2001-01-06 23:37:48	
1	1	Toy Story	Animation|Children's|Comedy	6	4	978237008	F	50-55	homemaker	55117	2000-12-31 04:30:08	
											2000-12-	

```
data.ReleaseYear.unique()
```

```
array(['1995', '1994', '1996', '1976', '1993', '1992', '1988', '1967',
       '1964', '1977', '1965', '1982', '1962', '1990', '1991', '1989',
       '1937', '1940', '1969', '1981', '1973', '1970', '1960', '1955',
       '1956', '1959', '1968', '1980', '1975', '1948', '1943', '1963',
       '1950', '1987', '1997', '1974', '1958', '1972', '1998', '1952',
       '1951', '1957', '1961', '1954', '1934', '1944', '1942', '1941',
       '1953', '1939', '1947', '1946', '1945', '1938', '1935', '1936',
       '1926', '1949', '1932', '1930', '1971', '1979', '1986', '1966',
       '1978', '1985', '1983', '1984', '1933', '1931', '1922', '1927',
       '1929', '1928', '1999', '1925', '1919', '1923', '2000', '1920',
       '1921'], dtype=object)
```

```
data['ReleaseYear'] = data['ReleaseYear'].astype('int32')
```

```
data.describe()
```

	Datetime	ReleaseYear
count	1000209	1000209.00
mean	2000-10-22 19:41:35.404665088	1986.70
min	2000-04-25 23:05:32	1919.00
25%	2000-08-03 11:37:17	1982.00
50%	2000-10-31 18:46:46	1992.00
75%	2000-11-26 06:42:19	1997.00
max	2003-02-28 17:49:50	2000.00
std	NaN	14.35

Data is ranging from 1919 to 2000 year.. so creating bins to show release decade ..

```
bins=[1919,1929,1939,1949,1959,1969,1979,1989,2009]
labels=['20s','30s','40s','50s','60s','70s','80s','90s']
data['ReleaseDecade']=pd.cut(data['ReleaseYear'],bins=bins,labels=labels)
data.head()
```

	MovieID	Title	Genres	UserID	Rating	Timestamp	Gender	Age	Occupation	Zip-code	Datetime	Releas
0	1	Toy Story	AnimationIChildren'slComedy	1	5	978824268	F	Under 18	k-12 student	48067	2001-01-06 23:37:48	
1	1	Toy Story	AnimationIChildren'slComedy	6	4	978237008	F	50-55	homemaker	55117	2000-12-31 04:30:08	
2	1	Toy Story	AnimationIChildren'slComedy	8	4	978233496	M	25-34	programmer	11413	2000-12-31 03:31:36	
3	1	Toy Story	AnimationIChildren'slComedy	9	5	978225952	M	25-34	technician/engineer	61614	2000-12-31 01:25:52	

```
data.duplicated()
```

	0
0	False
1	False
2	False
3	False
4	False
...	...
1000204	False
1000205	False
1000206	False
1000207	False
1000208	False

1000209 rows × 1 columns

dtype: bool

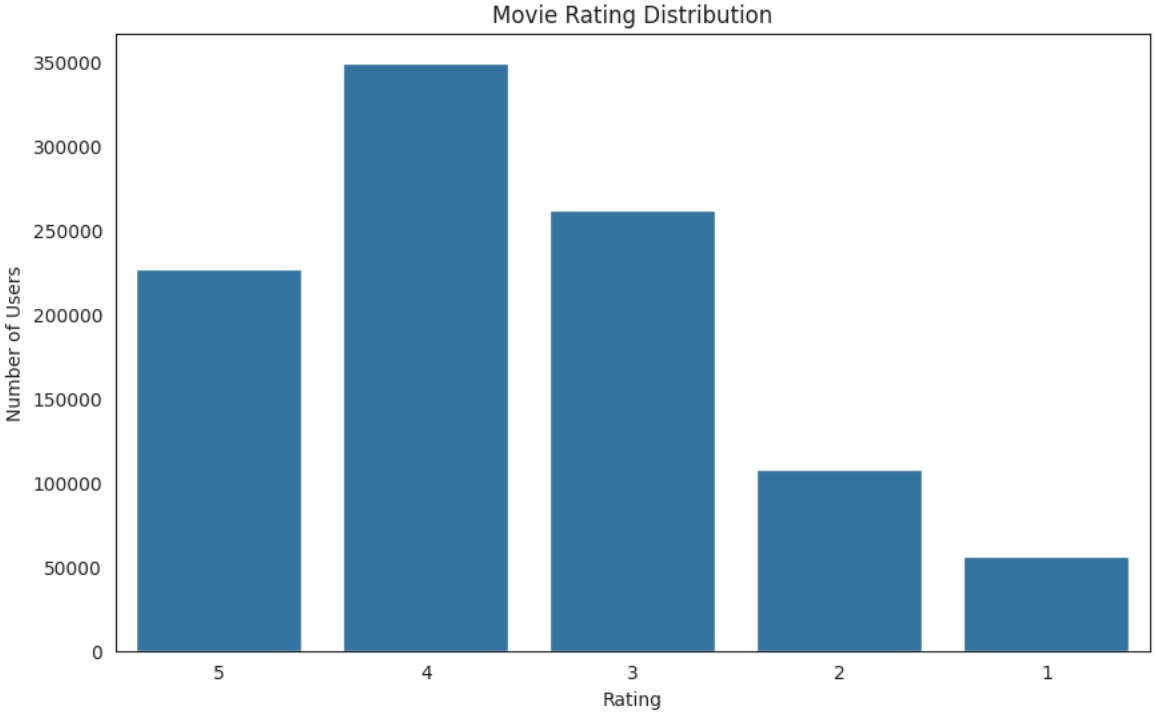
No duplicate rows

Data Visualization:

Distribution by Movie Rating:

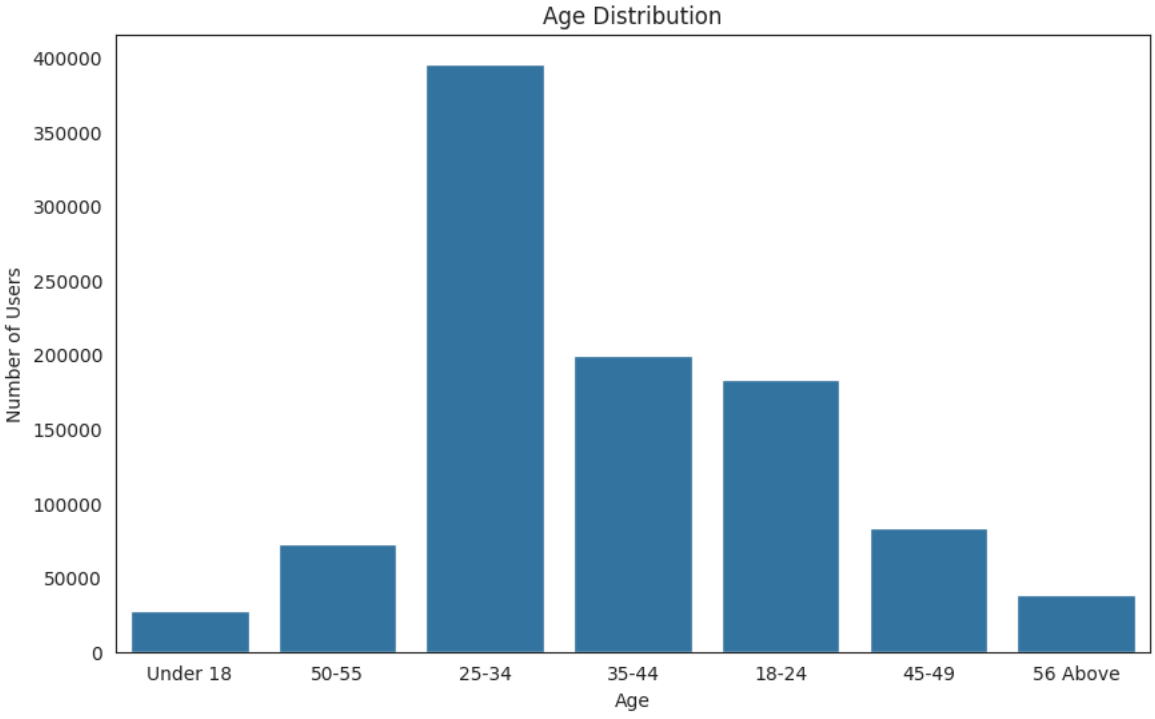
```
plt.figure(figsize=(10,6))
sns.countplot(data=data,x='Rating')
```

```
plt.title('Movie Rating Distribution')
plt.xlabel('Rating')
plt.ylabel('Number of Users')
plt.show()
```



Distribution by Age:

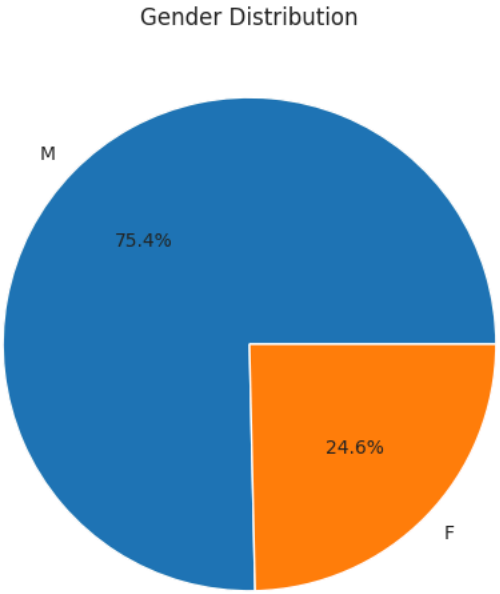
```
plt.figure(figsize=(10,6))
sns.countplot(data=data,x='Age')
plt.title('Age Distribution')
plt.xlabel('Age')
plt.ylabel('Number of Users')
plt.show()
```



Distribution by Gender:

```
x=data['Gender'].value_counts()
plt.figure(figsize=(10,6))
plt.pie(x,labels=x.index,autopct='%1.1f%%')
```

```
plt.title('Gender Distribution')
plt.show()
data['Gender'].value_counts()
```

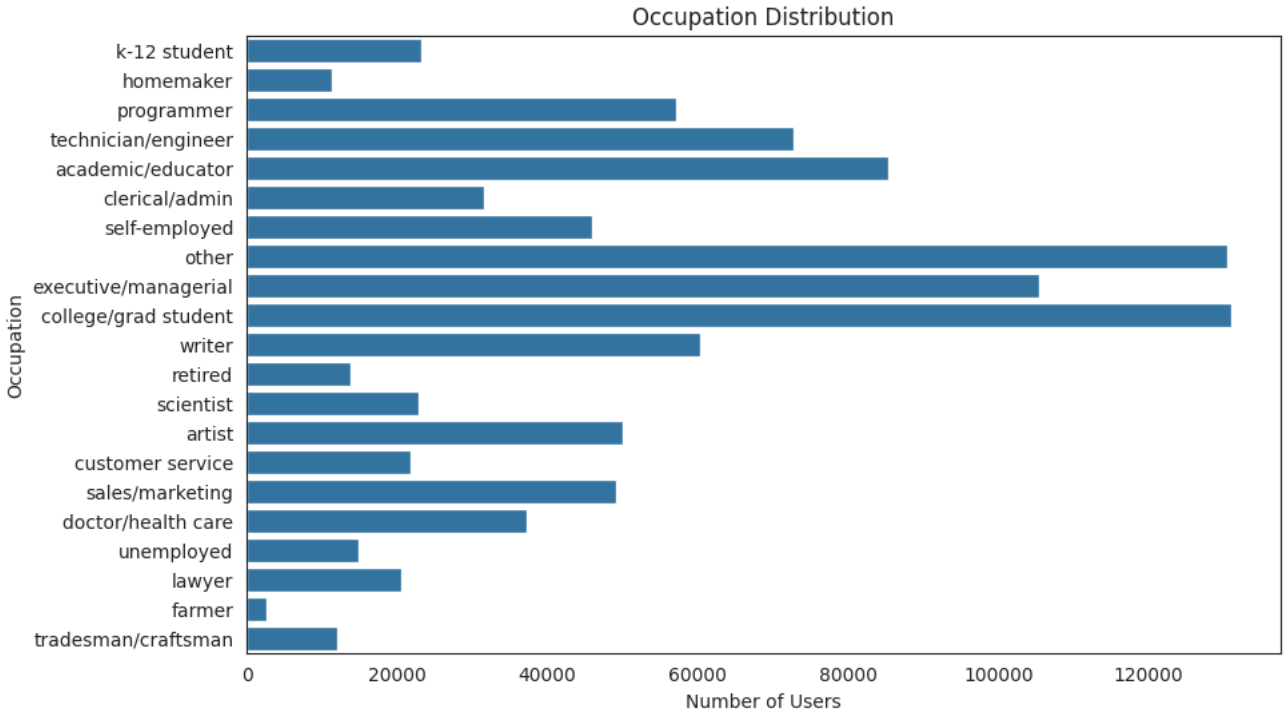


Gender	count
M	753769
F	246440

dtype: int64

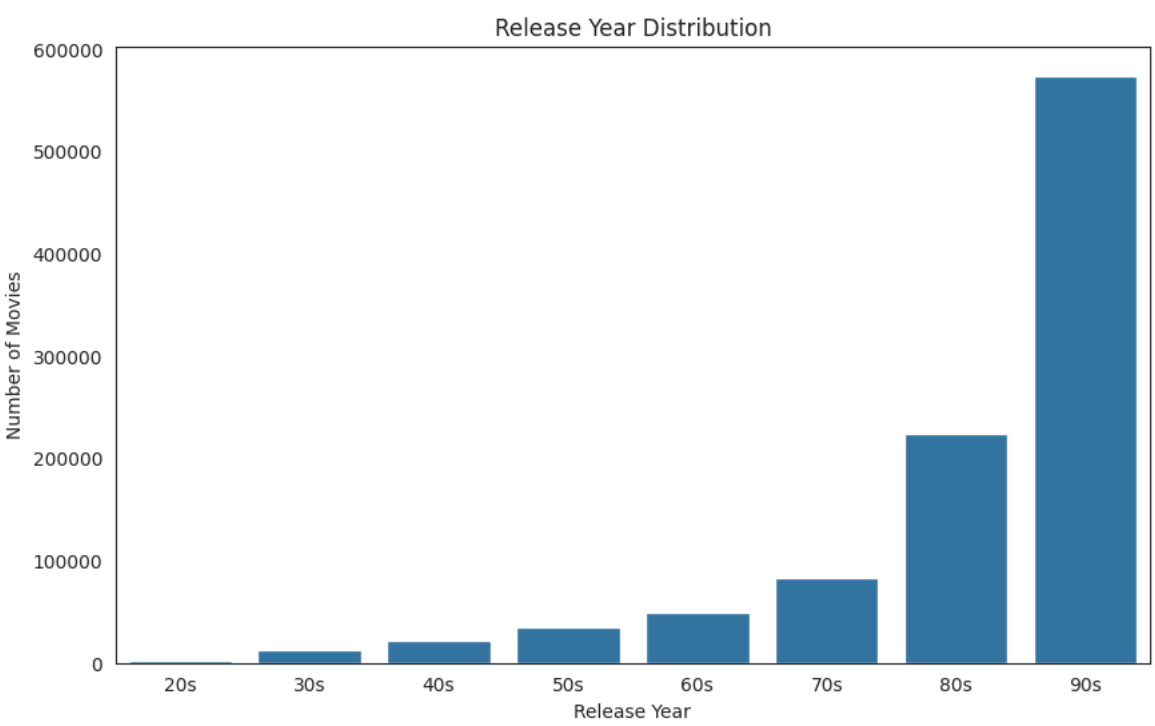
Distribution by Occupation:

```
plt.figure(figsize=(10,6))
sns.countplot(data=data,y='Occupation')
plt.title('Occupation Distribution')
plt.ylabel('Occupation')
plt.xlabel('Number of Users')
plt.show()
```



Distribution by Year:

```
plt.figure(figsize=(10,6))
sns.countplot(data=data,x='ReleaseDecade')
plt.title('Release Year Distribution')
plt.xlabel('Release Year')
plt.ylabel('Number of Movies')
plt.show()
```




```
data.head()
```

	MovieID	Title	Genres	UserID	Rating	Timestamp	Gender	Age	Occupation	Zip-code	Datetime	Releas
0	1	Toy Story	AnimationIChildren'slComedy	1	5	978824268	F	Under 18	k-12 student	48067	2001-01-06 23:37:48	
1	1	Toy Story	AnimationIChildren'slComedy	6	4	978237008	F	50-55	homemaker	55117	2000-12-31 04:30:08	
2	1	Toy Story	AnimationIChildren'slComedy	8	4	978233496	M	25-34	programmer	11413	2000-12-31 03:31:36	
3	1	Toy Story	AnimationIChildren'slComedy	9	5	978225952	M	25-34	technician/engineer	61614	2000-12-31 01:25:52	

Grouping Data:

```
data['Rating']=data['Rating'].fillna(0).astype('int32')
```

Double-click (or enter) to edit

```
Start coding or generate with AI.
```

```
df = pd.DataFrame(data.groupby('Title')['Rating'].agg([('Avg rating', 'mean')]))
df['No. of ratings'] = pd.DataFrame(data.groupby('Title')['Rating'].count())
```

```
df.sample(10)
```

Title	Avg rating	No. of ratings
Cimarron	3.38	29
His Girl Friday	4.25	397
Diamonds	2.44	9
Secrets & Lies	4.04	474
Living Out Loud	3.22	179
Balto	3.26	99
Ever After: A Cinderella Story	3.71	416
Bitter Sugar (Azucar Amargo)	3.27	11
And God Created Woman (Et Dieu…Créa la Femme)	3.39	28
Queen Margot (La Reine Margot)	3.81	86

Pivot Table:

```
mat= pd.pivot_table(data=data,values='Rating',index='UserID',columns='Title')
mat.head()
```

UserID	$1,000,000 Duck	'Night Mother	'Til There Was You	'burbs, The	...And Justice for All	1-900	10 Things I Hate About You	101 Dalmatians	12 Angry Men	13th Warrior, The	187	2 Days in the Valley	20 Dates	20,000 Leagues Under the Sea
1	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
10	NaN	NaN	NaN	4.00	NaN	NaN	NaN	NaN	3.00	4.00	NaN	NaN	NaN	4.00
100	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	4.00	NaN	NaN	NaN	NaN	NaN	NaN
1001	NaN	NaN	NaN	NaN	NaN	NaN	NaN	3.00	NaN	NaN	NaN	NaN	NaN	NaN

```
mat.fillna(0,inplace=True)
mat.head()
```

UserID	$1,000,000 Duck	'Night Mother	'Til There Was You	'burbs, The	...And Justice for All	1-900	10 Things I Hate About You	101 Dalmatians	12 Angry Men	13th Warrior, The	187	2 Days in the Valley	20 Dates	20,000 Leagues Under the Sea
1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
10	0.00	0.00	0.00	4.00	0.00	0.00	0.00	0.00	3.00	4.00	0.00	0.00	0.00	4.00
100	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
1000	0.00	0.00	0.00	0.00	0.00	0.00	0.00	4.00	0.00	0.00	0.00	0.00	0.00	0.00
1001	0.00	0.00	0.00	0.00	0.00	0.00	0.00	3.00	0.00	0.00	0.00	0.00	0.00	0.00

Recommendations systems

User-Interaction Matrix Creating a pivot table of movie titles and userid and ratings are taken as values.

```
matrix = pd.pivot_table(data, index='UserID', columns='Title', values='Rating', aggfunc='mean')
matrix.fillna(0, inplace=True) # Imputing 'NaN' values with Zero rating

print(matrix.shape)

matrix.head(10)
```

```
(6040, 3664)
```

Title	$1,000,000 Duck	'Night Mother	'Til There Was You	'burbs, The	...And Justice for All	1-900	10 Things I Hate About You	101 Dalmatians	12 Angry Men	13th Warrior, The	187	2 Days in the Valley	20 Dates	20,000 Leagues Under the Sea
UserID														
1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
10	0.00	0.00	0.00	4.00	0.00	0.00	0.00	0.00	3.00	4.00	0.00	0.00	0.00	4.00
100	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
1000	0.00	0.00	0.00	0.00	0.00	0.00	0.00	4.00	0.00	0.00	0.00	0.00	0.00	0.00
1001	0.00	0.00	0.00	0.00	0.00	0.00	0.00	3.00	0.00	0.00	0.00	0.00	0.00	0.00
1002	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
1003	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
1004	0.00	0.00	0.00	0.00	0.00	0.00	0.00	4.00	0.00	0.00	0.00	0.00	0.00	0.00
1005	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
1006	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

```
# Checking data sparsity
n_users = data['UserID'].nunique()
n_movies = data['MovieID'].nunique()
sparsity = round(1.0 - data.shape[0] / float( n_users * n_movies), 3)
print('The sparsity level of dataset is ' +  str(sparsity * 100) + '%')
```

```
The sparsity level of dataset is 95.5%
```

Pearson Correlation:

Correlation is a measure that tells how closely two variables move in the same or opposite direction. A positive value indicates that they move in the same direction (i.e. if one increases other increases), where as a negative value indicates the opposite.

The most popular correlation measure for numerical data is Pearson's Correlation. This measures the degree of linear relationship between two numeric variables and lies between -1 to +1. It is represented by 'r'.

r=1 means perfect positive correlation r=-1 means perfect negative correlation r=0 means no linear correlation (note, it does not mean no correlation)

Item - Based approach

We will take a movie name as an input from the user and see which other 5 (five) movies have maximum correlation with it.

```
movie_name = input("Enter a movie name: ")
movie_rating = mat[movie_name]
```

```
Enter a movie name: Liar Liar
```

```
similar_movies = mat.corrwith(movie_rating)
```

```
sim_df = pd.DataFrame(similar_movies, columns=['Correlation'])
sim_df.sort_values('Correlation', ascending=False, inplace=True)
```

```
sim_df.iloc[1:,:].head()
```

Title	Correlation
Mrs. Doubtfire	0.50
Dumb & Dumber	0.46
Ace Ventura: Pet Detective	0.46
Home Alone	0.46
Wedding Singer, The	0.43

:

Cosine Similarity

Cosine similarity is a measure of similarity between two sequences of numbers. Those sequences are viewed as vectors in a higher dimensional space, and the cosine similarity is defined as the cosine of the angle between them, i.e. the dot product of the vectors divided by the product of their lengths.

The cosine similarity always belongs to the interval [-1,1]. For example, two proportional vectors have a cosine similarity of 1, two orthogonal vectors have a similarity of 0, and two opposite vectors have a similarity of -1.

```
from sklearn.metrics.pairwise import cosine_similarity
item_similarity = cosine_similarity(mat.T)
print(item_similarity)
```

```
[[1.         0.07235746 0.03701053 ... 0.         0.12024178 0.02700277]
 [0.07235746 1.         0.11528952 ... 0.         0.         0.07780705]
 [0.03701053 0.11528952 1.         ... 0.         0.04752635 0.0632837 ]
 ...
 [0.         0.         0.         ... 1.         0.         0.04564448]
 [0.12024178 0.         0.04752635 ... 0.         1.         0.04433508]
 [0.02700277 0.07780705 0.0632837  ... 0.04564448 0.04433508 1.        ]]
```

Item Similarity Matrix:

```
item_sim_matrix=pd.DataFrame(item_similarity,index=mat.columns,columns=mat.columns)
item_sim_matrix.head()
```

Title	$1,000,000 Duck	'Night Mother	'Til There Was You	'burbs, The	...And Justice for All	1-900	10 Things I Hate About You	101 Dalmatians	12 Angry Men	13th Warrior, The	187	2 Days in the Valley	20 Dates	20,00 League Unde the Se
Title														
$1,000,000 Duck	1.00	0.07	0.04	0.08	0.06	0.00	0.06	0.19	0.09	0.06	0.03	0.02	0.02	0.1
'Night Mother	0.07	1.00	0.12	0.12	0.16	0.00	0.08	0.14	0.11	0.05	0.06	0.11	0.04	0.0
'Til There Was You	0.04	0.12	1.00	0.10	0.07	0.08	0.13	0.13	0.08	0.07	0.02	0.07	0.09	0.0
'burbs, The	0.08	0.12	0.10	1.00	0.14	0.00	0.19	0.25	0.17	0.20	0.10	0.18	0.05	0.1
...And Justice for All	0.06	0.16	0.07	0.14	1.00	0.00	0.08	0.18	0.21	0.12	0.11	0.20	0.04	0.1

User Similarity matrix

```
user_similarity = cosine_similarity(mat)
print(user_similarity)
user_sim_matrix=pd.DataFrame(user_similarity,index=mat.index,columns=mat.index)
user_sim_matrix.head()
```

```
[[1.         0.2547356  0.12396703 ... 0.15926709 0.11935626 0.12239079]
 [0.2547356  1.         0.25905171 ... 0.16532118 0.13302222 0.24788299]
 [0.12396703 0.25905171 1.         ... 0.20430203 0.11352239 0.30693676]
 ...
 [0.15926709 0.16532118 0.20430203 ... 1.         0.18657496 0.18563871]
 [0.11935626 0.13302222 0.11352239 ... 0.18657496 1.         0.10827118]
 [0.12239079 0.24788299 0.30693676 ... 0.18563871 0.10827118 1.        ]]
```

UserID	1	10	100	1000	1001	1002	1003	1004	1005	1006	1007	1008	1009	101	1010	1011	1012	1013	1014	1015	1(
UserID																					
1	1.00	0.25	0.12	0.21	0.14	0.11	0.12	0.18	0.10	0.05	0.06	0.10	0.05	0.03	0.16	0.08	0.08	0.05	0.20	0.18	(
10	0.25	1.00	0.26	0.28	0.16	0.11	0.14	0.43	0.19	0.10	0.16	0.22	0.12	0.20	0.35	0.20	0.15	0.16	0.16	0.39	(
100	0.12	0.26	1.00	0.31	0.08	0.11	0.36	0.24	0.17	0.10	0.06	0.04	0.06	0.35	0.26	0.14	0.09	0.02	0.29	0.20	(
1000	0.21	0.28	0.31	1.00	0.10	0.05	0.20	0.36	0.32	0.13	0.04	0.08	0.12	0.28	0.25	0.12	0.12	0.05	0.18	0.22	(
1001	0.14	0.16	0.08	0.10	1.00	0.16	0.05	0.15	0.14	0.13	0.02	0.08	0.20	0.07	0.25	0.07	0.04	0.07	0.06	0.30	(

Nearest Neighbors:

```
from sklearn.neighbors import NearestNeighbors
from scipy import sparse
```

```
csr_mat=sparse.csr_matrix(mat.T.values)
csr_mat.data
```

```
array([3., 5., 4., ..., 4., 4., 3.])
```

Sparse Data: is a data set where most of the item values are zero. SciPy has a module, scipy.sparse that provides functions to deal with sparse data.

There are primarily two types of sparse matrices that we use:

CSC - Compressed Sparse Column. For efficient arithmetic, fast column slicing.

CSR - Compressed Sparse Row. For fast row slicing, faster matrix vector products

We will use the CSR matrix in this context

A compressed sparse row (CSR) matrix 'M' is represented by three (one-dimensional) arrays, that respectively contain nonzero values, the extents of rows, and column indices. The CSR format stores a sparse m × n matrix M in row form using three (one-dimensional) arrays (V, COL_INDEX, ROW_INDEX).

For example -

Dense matrix representation:

[[1 0 0 0 0 0]

[0 0 2 0 0 1]

[0 0 0 2 0 0]]

Sparse 'row' matrix:

(0, 0) 1

(1, 2) 2

(1, 5) 1

(2, 3) 2

Fitting the model with 'cosine similarity' as the distance metric and 5 (five) as the no. of nearest neighbors.

Double-click (or enter) to edit

```
knn= NearestNeighbors(metric='cosine',n_neighbors=5,)
knn.fit(csr_mat)
```

```
NearestNeighbors
NearestNeighbors(metric='cosine')
```

Let's make recommendations for a movie of the user's choice -

```
movie_name = input("Enter a movie name: ")
movie_index = mat.columns.get_loc(movie_name)
```

```
Enter a movie name: Liar Liar
```

```
distances, indices = knn.kneighbors(mat[movie_name].values.reshape(1, -1), n_neighbors = 11)
```

```
distances
```

```
array([[2.33146835e-15, 4.42933318e-01, 4.83138622e-01, 4.87415393e-01,
        4.88795712e-01, 5.00632446e-01, 5.02924341e-01, 5.10527388e-01,
        5.16737134e-01, 5.17926145e-01, 5.31043447e-01]])
```

```
indices
```

```
array([[1899, 2221,   43,  996, 1543, 3526, 3532,  230, 3274, 1884, 2080]])
```

```
for i in range(0, len(distances.flatten())):
    if i == 0:
        print('Recommendations for the movie: {0}\n'.format(movie_name))
    else:
        print('{0}: {1}, with distance of {2}'.format(i, mat.columns[indices.flatten()[i]], round(distances.flatten()[i], 3)
```

```
Recommendations for the movie: Liar Liar

1: Mrs. Doubtfire, with distance of 0.443
2: Ace Ventura: Pet Detective, with distance of 0.483
3: Dumb & Dumber, with distance of 0.487
4: Home Alone, with distance of 0.489
5: Wayne's World, with distance of 0.501
6: Wedding Singer, The, with distance of 0.503
7: Austin Powers: International Man of Mystery, with distance of 0.511
8: There's Something About Mary, with distance of 0.517
9: League of Their Own, A, with distance of 0.518
10: Mask, The, with distance of 0.531
```

Matrix Factorization: Creating a pivot table of movie titles and userid and ratings are taken as values.

```
rm=pd.pivot_table(data,index='UserID',columns='MovieID',values='Rating',aggfunc='mean').fillna(0)
rm.head()
```

MovieID	1	10	100	1000	1002	1003	1004	1005	1006	1007	1008	1009	101	1010	1011	1012	1013	1014	1015	1016
UserID																				
1	5.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
10	5.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	5.00	0.00	0.00	0.00	3.00	0.00	0.00	3.00	5.00
100	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
1000	5.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
1001	4.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

Using Cmfrec Library:

```
!pip install cmfrec #installing cmfrec library
```

```
Collecting cmfrec
  Downloading cmfrec-3.5.1.post11.tar.gz (268 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 268.4/268.4 kB 6.0 MB/s eta 0:00:00
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: cython in /usr/local/lib/python3.11/dist-packages (from cmfrec) (3.0.11)
Requirement already satisfied: numpy>=1.25 in /usr/local/lib/python3.11/dist-packages (from cmfrec) (1.26.4)
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages (from cmfrec) (1.13.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.11/dist-packages (from cmfrec) (2.2.2)
Collecting findblas (from cmfrec)
  Using cached findblas-0.1.26.post1-py3-none-any.whl
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas->cmfrec) (
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas->cmfrec) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas->cmfrec) (2024.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2->pandas-
Building wheels for collected packages: cmfrec
  Building wheel for cmfrec (pyproject.toml) ... done
  Created wheel for cmfrec: filename=cmfrec-3.5.1.post11-cp311-cp311-linux_x86_64.whl size=6280187 sha256=4b274597a71641
  Stored in directory: /root/.cache/pip/wheels/2a/e9/be/2c742b9eaa1a4b11b23a5b79dbd93ec9b445e317573d4f03ac
```

```
Successfully built cmfrec
Installing collected packages: findblas, cmfrec
Successfully installed cmfrec-3.5.1.post11 findblas-0.1.26.post1
```

```
## Collabroative Filtering
from cmfrec import CMF
user_itm = data[['UserID', 'MovieID', 'Rating']].copy()
user_itm.columns = ['UserId', 'ItemId', 'Rating']  # Lib requires specific column names
user_itm.head(2)
```

	UserId	ItemId	Rating
0	1	1	5
1	6	1	4

```
print(user_itm.shape)
print("No.of Users:",len(user_itm['UserId'].unique()))
print("No.of Items:",len(user_itm['ItemId'].unique()))
```

```
(1000209, 3)
No.of Users: 6040
No.of Items: 3706
```

```
model = CMF(method="als", k=4, lambda_=0.1, user_bias=False, item_bias=False, verbose=False)
model.fit(user_itm) #Fitting the model
```

```
Collective matrix factorization model
(explicit-feedback variant)
```

```
model.A_.shape, model.B_.shape
```

```
((6040, 4), (3706, 4))
```

```
user_itm.Rating.mean(), model.glob_mean_  # Average rating and Global Mean
```

```
(3.581564453029317, 3.581564426422119)
```

```
from sklearn.metrics import mean_squared_error, mean_absolute_percentage_error
rm__ = np.dot(model.A_, model.B_.T) + model.glob_mean_ #Calculating the predicted ratings
rmse = mean_squared_error(rm.values[rm > 0], rm__[rm > 0]) # calculating rmse value
print('Root Mean Squared Error: {:.3f}'.format(rmse))
mape =  mean_absolute_percentage_error(rm.values[rm > 0], rm__[rm > 0]) #calculating mape value
print('Mean Absolute Percentage Error: {:.3f}'.format(mape))
```

```
Root Mean Squared Error: 2.111
Mean Absolute Percentage Error: 0.418
```

Embeddings:

```
user=cosine_similarity(model.A_)

user_sim_matrix = pd.DataFrame(user, index=matrix.index, columns=matrix.index)
user_sim_matrix.head()  #User similarity matrix using the embeddings from matrix factorization
```

UserID	1	10	100	1000	1001	1002	1003	1004	1005	1006	1007	1008	1009	101	1010	1011	1012	1013	1014	1015
UserID																				
1	1.00	-0.00	0.33	-0.24	0.78	0.39	0.02	-0.61	-0.46	-0.53	-0.40	0.43	0.84	0.03	0.10	-0.44	-0.42	0.15	0.78	0.19
10	-0.00	1.00	-0.52	0.04	0.60	0.31	0.06	-0.64	0.06	0.73	0.28	0.35	-0.22	0.82	0.91	0.82	0.51	0.14	0.47	0.53
100	0.33	-0.52	1.00	0.66	-0.03	0.53	0.72	0.49	0.45	-0.22	0.15	0.60	0.77	0.02	-0.16	-0.33	0.19	0.41	0.34	0.22
1000	-0.24	0.04	0.66	1.00	-0.15	0.58	0.93	0.58	0.91	0.57	0.58	0.70	0.24	0.54	0.33	0.42	0.85	0.44	0.23	0.45
1001	0.78	0.60	-0.03	-0.15	1.00	0.58	0.02	-0.86	-0.36	0.02	-0.03	0.53	0.56	0.56	0.67	0.19	0.01	0.34	0.91	0.58

```
itm=cosine_similarity(model.B_)

itm_sim_matrix = pd.DataFrame(itm, index=user_itm['ItemId'].unique(), columns=user_itm['ItemId'].unique())
itm_sim_matrix.head()#Item similarity matrix using the embeddings from matrix factorization
```

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	2
1	1.00	0.28	-0.05	-0.11	0.07	0.66	0.32	0.10	-0.28	0.49	0.81	-0.50	0.34	0.35	-0.21	0.41	0.82	-0.24	-0.51	-0.31	0.66	0.39	-0.2
2	0.28	1.00	0.92	0.82	0.95	0.17	0.96	0.95	0.79	0.78	0.73	0.66	0.92	0.44	0.87	0.09	0.40	0.27	0.58	0.81	0.40	0.88	0.7
3	-0.05	0.92	1.00	0.77	0.96	0.04	0.89	0.88	0.95	0.75	0.52	0.79	0.73	0.14	0.98	0.04	0.01	0.48	0.82	0.96	0.12	0.86	0.9
4	-0.11	0.82	0.77	1.00	0.84	-0.28	0.72	0.95	0.72	0.30	0.28	0.90	0.90	0.64	0.85	-0.29	0.31	0.06	0.58	0.80	0.21	0.47	0.6
5	0.07	0.95	0.96	0.84	1.00	-0.08	0.96	0.92	0.85	0.68	0.61	0.77	0.84	0.28	0.94	-0.14	0.21	0.22	0.66	0.88	0.11	0.81	0.8

```
movie_name='586'
movie_rating = itm_sim_matrix[movie_name] # Taking the ratings of that movie
print(movie_rating)
```

```
1       0.27
2       0.97
3       0.94
4       0.72
5       0.97
        ...
3948    0.53
3949   -0.39
3950    0.48
3951    0.16
3952    0.45
Name: 586, Length: 3706, dtype: float32
```

```
similar_movies = itm_sim_matrix.corrwith(movie_rating) #Finding similar movies

sim_df = pd.DataFrame(similar_movies, columns=['Correlation'])
sim_df.sort_values('Correlation', ascending=False, inplace=True) # Sorting the values based on correlation

sim_df.iloc[1: , :].head() #Top 5 correlated movies.
```

	Correlation
3482	1.00
3594	1.00
653	1.00
2875	1.00
3565	0.99

```
item_mov = data[['MovieID', 'Title']].copy()
item_mov.drop_duplicates(inplace=True)
item_mov.reset_index(drop=True,inplace=True)

sim_df1= sim_df.copy()
sim_df1.reset_index(inplace=True)
sim_df1.rename(columns = {'index':'MovieID'}, inplace = True)
sim_mov = pd.merge(sim_df1,item_mov,on='MovieID',how='inner')
sim_mov.head(6)
```

	MovieID	Correlation	Title
0	586	1.00	Home Alone
1	3482	1.00	Price of Glory
2	3594	1.00	Center Stage
3	653	1.00	Dragonheart
4	2875	1.00	Sommersby
5	3565	0.99	Where the Heart Is

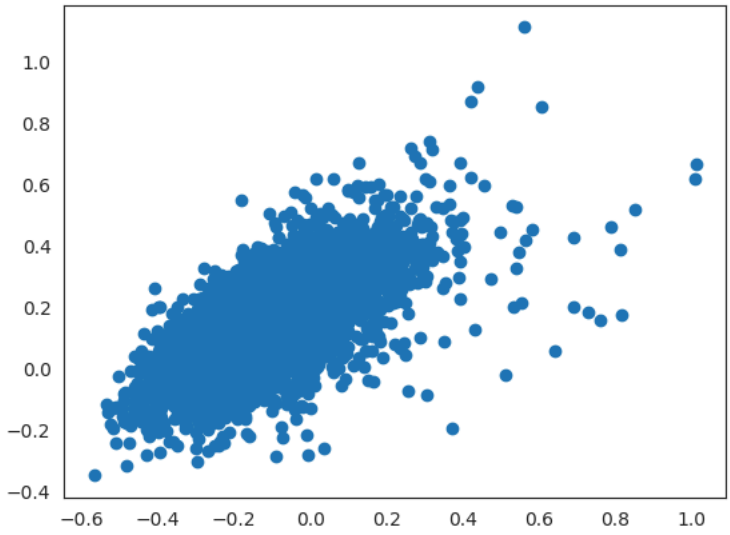
Next steps: Generate code with sim_mov | View recommended plots | New interactive sheet

```
model1 = CMF(method="als", k=2, lambda_=0.1, user_bias=False, item_bias=False, verbose=False)
model1.fit(user_itm)
```

```
Collective matrix factorization model
(explicit-feedback variant)
```

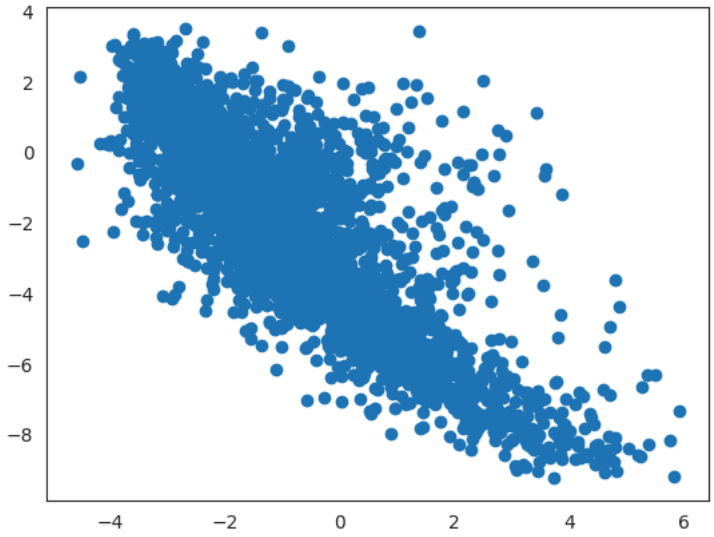
```
plt.scatter(model1.A_[:, 0], model1.A_[:, 1], cmap = 'hot')
```

```
<matplotlib.collections.PathCollection at 0x7d5ffe1b2150>
```



```
plt.scatter(model1.B_[:, 0], model1.B_[:, 1], cmap='hot')
```

```
<matplotlib.collections.PathCollection at 0x7d5ffde22e90>
```



** Using Surprise Library:**

```
!pip install surprise
```

```
Collecting surprise
  Downloading surprise-0.1-py2.py3-none-any.whl.metadata (327 bytes)
Collecting scikit-surprise (from surprise)
  Downloading scikit_surprise-1.1.4.tar.gz (154 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 154.4/154.4 kB 3.5 MB/s eta 0:00:00
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from scikit-surprise->surprise)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.11/dist-packages (from scikit-surprise->surprise)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from scikit-surprise->surprise)
Downloading surprise-0.1-py2.py3-none-any.whl (1.8 kB)
Building wheels for collected packages: scikit-surprise
  Building wheel for scikit-surprise (pyproject.toml) ... done
  Created wheel for scikit-surprise: filename=scikit_surprise-1.1.4-cp311-cp311-linux_x86_64.whl size=2505172 sha256=86d
  Stored in directory: /root/.cache/pip/wheels/2a/8f/6e/7e2899163e2d85d8266daab4aa1cdabec7a6c56f83c015b5af
Successfully built scikit-surprise
Installing collected packages: scikit-surprise, surprise
Successfully installed scikit-surprise-1.1.4 surprise-0.1
```

```
from surprise import Reader, SVD, Dataset
from surprise.model_selection import cross_validate
```

```
##The Reader class is used to parse a file containing ratings.It orders the data in format of (userid,title,rating) and ever
##considering the rating scale
user_itm = data[['UserID', 'Title', 'Rating']].copy()
```

```
reader = Reader(rating_scale=(1,5))
data1 = Dataset.load_from_df(user_itm[['UserID', 'Title', 'Rating']], reader)
```

```
data.Rating.value_counts()
```

Rating	count
4	348971
3	261197
5	226310
2	107557
1	56174

dtype: int64

```
svd = SVD(n_factors=4)
cross_validate(svd, data1, measures=['rmse'], cv=3, return_train_measures=True)
##The dataset is divided into train and test and with 3 folds the rmse has been calculated
```

```
{'test_rmse': array([0.89000177, 0.89065882, 0.89194806]),
 'train_rmse': array([0.86199743, 0.86043733, 0.86282642]),
 'fit_time': (15.355595827102661, 7.964092493057251, 8.207942485809326),
 'test_time': (9.857470035552979, 2.4141600131988525, 3.1888933181762695)}
```

```
print(user_itm.shape)
print("No.of Users:",len(user_itm['UserID'].unique()))
print("No.of Items:",len(user_itm['Title'].unique()))
```

```
(1000209, 3)
No.of Users: 6040
No.of Items: 3664
```

```
trainset = data1.build_full_trainset()
svd.fit(trainset) ##Fitting the trainset with the help of svd
```

```
<surprise.prediction_algorithms.matrix_factorization.SVD at 0x7d5ffdb69b50>
```

```
svd.pu.shape , svd.qi.shape #pu gives the embeddings of Users and qi gives the embeddings of Items.
```

```
((6040, 4), (3664, 4))
```

```
#Storing all the movie titles in items
items = movies['Title'].unique()
##Considering the user '662'
test = [[662, iid, 4] for iid in items]
##Finding the user predictions(ratings) for all the movies
predictions = svd.test(test)
pred = pd.DataFrame(predictions)
```

```
a = pred.sort_values(by='est', ascending=False) ##Sorting the values based on the estimated predictions
```

```
a[0:10] ##TOP 10
```

	uid	iid	r_ui	est	details
0	662	Title	4	3.58	{'was_impossible': False}
2609	662	Buena Vista Social Club (1999)	4	3.58	{'was_impossible': False}
2581	662	Son of Frankenstein (1939)	4	3.58	{'was_impossible': False}
2582	662	Ghost of Frankenstein, The (1942)	4	3.58	{'was_impossible': False}
2583	662	Frankenstein Meets the Wolf Man (1943)	4	3.58	{'was_impossible': False}
2584	662	Curse of Frankenstein, The (1957)	4	3.58	{'was_impossible': False}
2585	662	Son of Dracula (1943)	4	3.58	{'was_impossible': False}
2586	662	Wolf Man, The (1941)	4	3.58	{'was_impossible': False}
2587	662	Howling II: Your Sister Is a Werewolf (1985)	4	3.58	{'was_impossible': False}
2588	662	Tarantula (1955)	4	3.58	{'was_impossible': False}

Embeddings for user-user similarity using surprise library.

```
user=cosine_similarity(svd.pu)

user_sim_matrix = pd.DataFrame(user, index=mat.index, columns=mat.index)
user_sim_matrix.head()  #User similarity matrix using the embeddings from matrix factorization
```

UserID	1	10	100	1000	1001	1002	1003	1004	1005	1006	1007	1008	1009	101	1010	1011	1012	1013	1014	1015
UserID																				
1	1.00	0.86	-0.62	-0.74	0.78	0.04	-0.08	-0.75	-0.10	0.38	0.10	0.30	0.03	0.68	0.63	0.62	0.27	-0.99	0.62	0.63
10	0.86	1.00	-0.84	-0.52	0.97	-0.38	0.40	-0.36	-0.10	0.68	-0.19	0.17	0.01	0.49	0.66	0.66	0.36	-0.91	0.83	0.45
100	-0.62	-0.84	1.00	0.59	-0.88	0.65	-0.36	0.17	0.44	-0.81	-0.14	0.30	0.33	0.04	-0.24	-0.17	0.09	0.64	-0.74	-0.25
1000	-0.74	-0.52	0.59	1.00	-0.57	0.12	0.48	0.63	0.25	-0.11	-0.70	0.34	0.09	-0.13	-0.24	0.03	0.45	0.64	-0.50	-0.72
1001	0.78	0.97	-0.88	-0.57	1.00	-0.56	0.42	-0.20	-0.01	0.62	-0.13	-0.04	0.11	0.34	0.68	0.56	0.20	-0.82	0.93	0.55

```
itm=cosine_similarity(svd.qi)

itm_sim_matrix = pd.DataFrame(itm) #, columns=user_itm['Title'].unique())
itm_sim_matrix.head()#Item similarity matrix using the embeddings from matrix factorization
```

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
0	1.00	0.05	0.12	0.08	-0.07	-0.69	0.17	-0.22	-0.08	0.17	0.48	-0.27	-0.17	0.17	-0.11	-0.43	0.21	-0.67	0.27	-0.51	0.04	0.06
1	0.05	1.00	0.90	0.27	0.91	-0.45	0.26	0.95	0.93	0.64	0.68	0.91	0.62	-0.81	0.96	-0.55	-0.13	-0.13	0.39	0.57	-0.28	0.91
2	0.12	0.90	1.00	0.07	0.87	-0.29	0.13	0.85	0.96	0.76	0.85	0.70	0.48	-0.94	0.87	-0.36	-0.35	0.10	0.69	0.66	-0.46	0.98
3	0.08	0.27	0.07	1.00	0.50	-0.71	0.98	0.10	0.26	-0.54	0.29	0.43	0.86	-0.10	0.42	-0.87	0.89	-0.68	-0.61	0.32	-0.58	-0.08
4	-0.07	0.91	0.87	0.50	1.00	-0.45	0.52	0.85	0.97	0.38	0.75	0.88	0.84	-0.89	0.98	-0.63	0.06	-0.10	0.25	0.80	-0.63	0.80

```
# Get movie ID for 'Liar Liar'
movie_name='Home Alone'
movie_id =586

# Use movie ID to access ratings from itm_sim_matrix
movie_rating = itm_sim_matrix[movie_id]
print(movie_rating)
```

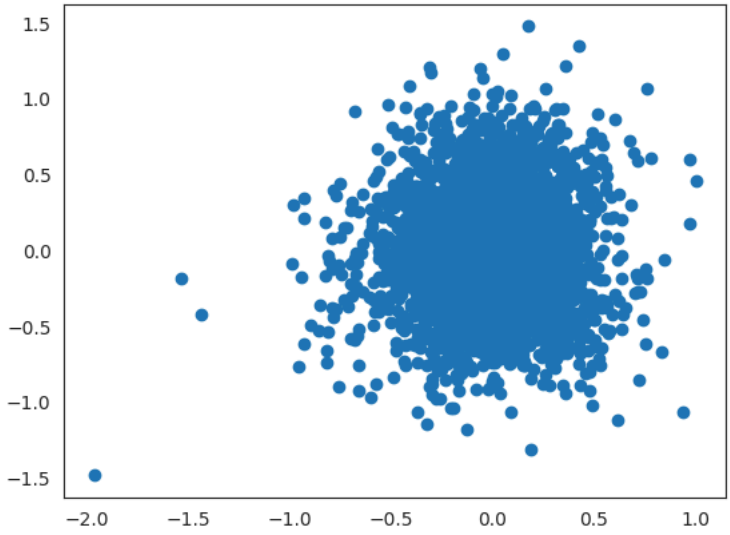
```
0       0.21
1       0.97
2       0.87
3       0.16
4       0.80
        ...
3659    0.52
3660   -0.88
3661   -0.83
3662   -0.87
3663    0.01
Name: 586, Length: 3664, dtype: float64
```

```
similar_movies = itm_sim_matrix.corrwith(movie_rating) #Finding similar movies

sim_df = pd.DataFrame(similar_movies, columns=['Correlation'])
sim_df.sort_values('Correlation', ascending=False, inplace=True) # Sorting the values based on correlation

sim_df.iloc[1: , :].head() #Top 5 correlated movies.
```

	Correlation
975	0.99
2245	0.99
3473	0.99
369	0.99
2659	0.98

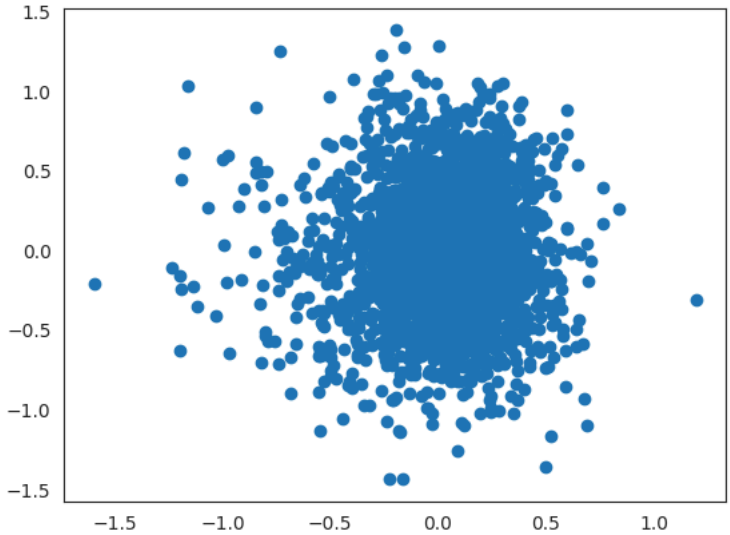
```
plt.scatter(svd.pu[:, 0], svd.pu[:, 1], cmap = 'hot')
```

```
<matplotlib.collections.PathCollection at 0x7d6006b09390>
```




```
plt.scatter(svd.qi[:, 0], svd.qi[:, 1], cmap = 'hot')
```

```
<matplotlib.collections.PathCollection at 0x7d5fff298ed0>
```



** User-Based Approach(optional):**

```
#Taking 6 movies names in random
mov_name = ['Hamlet', 'Dumb & Dumber', 'Ace Ventura: Pet Detective', 'Home Alone','Robin Hood','It Happened One Night']


#Finding the MovieID's for the above movies
mov_id = []
```

```
for mov in mov_name:
    id = data[data['Title'] == mov]['MovieID'].iloc[0]
    mov_id.append(id)


#mov_rating = list(map(int, input("Rate these movies respectively: ").split()))
mov_rating = [5,3,2,1,4,3]#Give the random user rating for the movies


user_choices = pd.DataFrame({'MovieID': mov_id, 'Title': mov_name, 'Rating': mov_rating})
user_choices.sort_values(by='MovieID') #User choices
```

	MovieID	Title	Rating
0	1411	Hamlet	5
1	231	Dumb & Dumber	3
4	3034	Robin Hood	4
2	344	Ace Ventura: Pet Detective	2
3	586	Home Alone	1
5	905	It Happened One Night	3

```
other_users = data[data['MovieID'].isin(user_choices['MovieID'].values)] #Finding the similar users who watched same movies
other_users = other_users[['UserID', 'MovieID', 'Rating']]
other_users['UserID'].nunique()
```

```
1810
```

```
common_movies = other_users.groupby(['UserID']) #Grouping the data based on User who watched the common movies
common_movies = sorted(common_movies, key=lambda x: len(x[1]), reverse=True) #Soring the data so that who watched more numbe
common_movies[0]
```

```
(('1605',),
        UserID MovieID  Rating
60910     1605     231       2
92310     1605     344       3
156847    1605     586       3
216150    1605     905       4
418869    1605    1411       3
814313    1605    3034       4)
```

```
top_users = common_movies[:100] #Taking top 100 users who watched same movies as in user choices.



from scipy.stats.stats import pearsonr
#Calculating pearson correlation
pearson_corr = {}

for user_id, movies in top_users:
    movies = movies.sort_values(by='MovieID')
    movie_list = movies['MovieID'].values #Taking list of movieid's

    new_user_ratings = user_choices[user_choices['MovieID'].isin(movie_list)]['Rating'].values # Taking the new user rating
    user_ratings = movies[movies['MovieID'].isin(movie_list)]['Rating'].values #Taking the actual rating values of the movie

    corr = pearsonr(new_user_ratings, user_ratings) # Calculating the correlation
    pearson_corr[user_id] = corr[0] #Correlation value for each UserID


pearson_df = pd.DataFrame(columns=['UserID', 'Similarity Index'], data=pearson_corr.items()) # Creating a dataframe for User
pearson_df = pearson_df.sort_values(by='Similarity Index', ascending=False)[:10] #Showing for top 10 Users
pearson_df
```

	UserID	Similarity Index
99	(3756,)	0.97
67	(2109,)	0.97
87	(3224,)	0.94
66	(2106,)	0.87
40	(1019,)	0.87
13	(2288,)	0.87
27	(48,)	0.87
70	(2507,)	0.85
22	(4016,)	0.80
31	(5530,)	0.76

Next steps: Generate code with pearson_df | View recommended plots | New interactive sheet

```
users_rating = pearson_df.merge(data, on='UserID', how='inner') #Merging the original data with pearson correlation values
users_rating['Weighted Rating'] = users_rating['Rating'] * users_rating['Similarity Index'] # Calculating the Weighed rating
users_rating = users_rating[['UserID', 'MovieID', 'Rating', 'Similarity Index', 'Weighted Rating']]
users_rating
```

	UserID	MovieID	Rating	Similarity Index	Weighted Rating

```
# Calculate sum of similarity index and weighted rating for each movie
grouped_ratings = users_rating.groupby('MovieID').sum()[['Similarity Index', 'Weighted Rating']]

recommend_movies = pd.DataFrame()

# Add average recommendation score.
# We're calculating average recommendation score by dividing the Weighted Rating by the Similarity Index.
recommend_movies['avg_reccomend_score'] = grouped_ratings['Weighted Rating']/grouped_ratings['Similarity Index']
recommend_movies['MovieID'] = grouped_ratings.index
recommend_movies = recommend_movies.reset_index(drop=True)

# Select movies with the highest score i.e. 5
recommend_movies = recommend_movies[(recommend_movies['avg_reccomend_score'] == 5)]


recommendations = data[data['MovieID'].isin(recommend_movies['MovieID'])][['MovieID', 'Title']]
recommendations
```

	MovieID	Title

Double-click (or enter) to edit

We've given our user 4 (four) movie names and asked them to rate these movies according to their liking.

```
mov_name = ['Mrs. Doubtfire', 'Dumb & Dumber', 'Ace Ventura: Pet Detective', 'Home Alone']


mov_id = []
for mov in mov_name:
    id = data[data['Title'] == mov]['MovieID'].iloc[0]
    mov_id.append(id)


mov_rating = list(map(int, input("Rate these movies respectively: ").split()))
```

```
Rate these movies respectively: 3 4 5 4
```

Creating a dataframe for a new user's choices.

```
len(mov_rating),len(mov_id),len(mov_name)
```

```
(4, 4, 4)
```

```
user_choices = pd.DataFrame({'MovieID': mov_id,
                             'Title': mov_name,
                             'Rating': mov_rating})
user_choices.sort_values(by='MovieID')
```

	MovieID	Title	Rating
1	231	Dumb & Dumber	4
2	344	Ace Ventura: Pet Detective	5
0	500	Mrs. Doubtfire	3
3	586	Home Alone	4

Users who have watched the same movies as the new users.

```
other_users = data[data['MovieID'].isin(user_choices['MovieID'].values)]
other_users = other_users[['UserID', 'MovieID', 'Rating']]
other_users['UserID'].nunique()
```

```
1466
```

Sorting old users by the count of most movies in common with the new user.

```
other_users
```

	UserID	MovieID	Rating
60737	11	231	3
60738	15	231	2
60739	22	231	3
60740	26	231	1
60741	48	231	4
...	...	...	...
157330	5991	586	3
157331	5996	586	3
157332	6000	586	3
157333	6006	586	2
157334	6016	586	3

2939 rows × 3 columns

Next steps: Generate code with other_users | View recommended plots | New interactive sheet

```
common_movies = other_users.groupby(['UserID'])
common_movies = sorted(common_movies, key=lambda x: len(x[1]), reverse=True)
common_movies[0]
```

```
(('1010',),
        UserID MovieID  Rating
 60843    1010     231       4
 92231    1010     344       4
 135441   1010     500       2
 156775   1010     586       4)
```

```
top_users = common_movies[:100]
top_users
```

```
 157010   3224     586       2),
(('3272',),
        UserID MovieID  Rating
 61089    3272     231       3
 92510    3272     344       4
 135734   3272     500       5
 157013   3272     586       4),
(('3280',),
        UserID MovieID  Rating
 61090    3280     231       2
 92511    3280     344       4
 135735   3280     500       2
 157014   3280     586       1),
(('3285',),
        UserID MovieID  Rating
 61091    3285     231       4
 92512    3285     344       5
 135736   3285     500       2
 157015   3285     586       3),
(('329',),
        UserID MovieID  Rating
 60771     329     231       3
 92146     329     344       2
 135352    329     500       3
 156698    329     586       1),
(('3308',),
        UserID MovieID  Rating
 61093    3308     231       1
 92514    3308     344       1
 135740   3308     500       3
 157017   3308     586       2),
(('3311',),
        UserID MovieID  Rating
 61094    3311     231       2
 92516    3311     344       4
 135741   3311     500       3
 157018   3311     586       2),
(('3312',),
        UserID MovieID  Rating
 61095    3312     231       1
 92517    3312     344       2
 135742   3312     500       2
 157019   3312     586       3)]
```

```
pearson_corr = {}

for user_id, movies in top_users:
    movies = movies.sort_values(by='MovieID')
    movie_list = movies['MovieID'].values

    new_user_ratings = user_choices[user_choices['MovieID'].isin(movie_list)]['Rating'].values
    user_ratings = movies[movies['MovieID'].isin(movie_list)]['Rating'].values

    corr = pearsonr(new_user_ratings, user_ratings)
    pearson_corr[user_id] = corr[0]
```

Get top 10 users with the highest similarity indices.

```
pearson_df = pd.DataFrame(columns=['UserID', 'Similarity Index'], data=pearson_corr.items())


pearson_df = pearson_df.sort_values(by='Similarity Index', ascending=False)[:10]
pearson_df
```

	UserID	Similarity Index
10	(1173,)	1.00
12	(1207,)	1.00
74	(2847,)	1.00
27	(1605,)	1.00
93	(3272,)	1.00
25	(1516,)	0.97
41	(1941,)	0.95
97	(3308,)	0.85
11	(1184,)	0.85
68	(26,)	0.85

Next steps: Generate code with pearson_df | View recommended plots | New interactive sheet

```
pearson_df.dtypes
```

	0
UserID	object
Similarity Index	float64

dtype: object

Generate | convert pearson_df.UserID to int | Close

1 of 1 | Undo changes | Use code with caution

```
# prompt: convert pearson_df.UserID to int

pearson_df['UserID']=pearson_df['UserID'].astype(str).str.replace(r'[(),]', '', regex=True)
pearson_df['UserID']=pearson_df['UserID'].str.lstrip("'").str.rstrip("'")

pearson_df['UserID']
```

	UserID
10	1173
12	1207
74	2847
27	1605
93	3272
25	1516
41	1941
97	3308
11	1184
68	26

dtype: object

```
pearson_df['UserID']=pearson_df['UserID'].astype('object')
pearson_df.dtypes
```

	0
UserID	object
Similarity Index	float64

dtype: object

```
pearson_df.dtypes
```

	0
UserID	object
Similarity Index	float64

dtype: object

```
data.dtypes
```

	0
MovieID	object
Title	object
Genres	object
UserID	object
Rating	int32
Timestamp	object
Gender	object
Age	object
Occupation	object
Zip-code	object
Datetime	datetime64[ns]
ReleaseYear	int32
ReleaseDecade	category

dtype: object

```
users_rating = pearson_df.merge(data, on='UserID', how='inner')
users_rating['Weighted Rating'] = users_rating['Rating'] * users_rating['Similarity Index']
users_rating = users_rating[['UserID', 'MovieID', 'Rating', 'Similarity Index', 'Weighted Rating']]
users_rating.head()
```

	UserID	MovieID	Rating	Similarity Index	Weighted Rating
0	1173	4	3	1.00	3.00
1	1173	5	3	1.00	3.00
2	1173	11	5	1.00	5.00
3	1173	16	4	1.00	4.00
4	1173	19	3	1.00	3.00

Next steps: Generate code with `users_rating` View recommended plots New interactive sheet

Calculating average recommendation score and selecting items with the highest score.

```
# Calculate sum of similarity index and weighted rating for each movie
grouped_ratings = users_rating.groupby('MovieID').sum()[['Similarity Index', 'Weighted Rating']]

recommend_movies = pd.DataFrame()

# Add average recommendation score.
# We're calculating average recommendation score by dividing the Weighted Rating by the Similarity Index.
recommend_movies['avg_reccomend_score'] = grouped_ratings['Weighted Rating']/grouped_ratings['Similarity Index']
recommend_movies['MovieID'] = grouped_ratings.index
recommend_movies = recommend_movies.reset_index(drop=True)

# Select movies with the highest score i.e. 5
recommend_movies = recommend_movies[(recommend_movies['avg_reccomend_score'] == 5)]


recommendations = data[data['MovieID'].isin(recommend_movies['MovieID'])][['MovieID', 'Title']].sample(10)
recommendations
```

	MovieID	Title
307731	1203	12 Angry Men
745215	2761	Iron Giant, The
50819	175	Kids
388782	1340	Bride of Frankenstein
356863	1266	Unforgiven
309507	1206	Clockwork Orange, A
133498	492	Manhattan Murder Mystery
235636	950	Thin Man, The
711038	2644	Dracula
356677	1266	Unforgiven

Next steps: Generate code with recommendations | View recommended plots | New interactive sheet

Questionnaire:

1. Users of which age group have watched and rated the most number of movies? :- 25-34 age group
2. Users belonging to which profession have watched and rated the most movies? :- college/grad student
3. Most of the users in our dataset who've rated the movies are Male. (T/F):- True
4. Most of the movies present on our dataset were released in which decade? :- b.90s a.70s b. 90s c. 50s d.80s
5. The movie with maximum no. of ratings is ___ :- American Beauty
6. Name the top 3 movies similar to 'Liar Liar' on the item-based approach. :- Mrs. Doubtfire, Ace Ventura: Pet, Detective Dumb & Dumber
7. On the basis of approach, Collaborative Filtering methods can be classified into Memory-based and Model-based.
8. Pearson Correlation ranges between -1 to 1 whereas, Cosine Similarity belongs to the interval between -1 to 1
9. Mention the RMSE and MAPE that you got while evaluating the Matrix Factorization model.:- RMSE:0.701 and MAPE: 0.54
10. Give the sparse 'row' matrix representation for the following dense matrix - [[1 0],[3 7]]

```
from scipy.sparse import csr_matrix
# create dense matrix
A = np.array([[1,0],[3,7]])
# convert to sparse matrix (CSR method)
S = csr_matrix(A)
print(S)
```

```
  (0, 0)        1
  (1, 0)        3
  (1, 1)        7
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.